Assignment 3: Multi-Agent System using the A2A Protocol

Trainer: Umashankar K H · Client: Innomatics

Build a small team of AI agents that talk to each other using the **A2A (Agent2Agent) protocol**, where each agent owns one job and uses tools to work on real data. You will get one of four sets (A, B, C or D). Each has its own data, its own three agents and its own business question.

Before you start

A2A in plain words

Word	What it means for you
Agent Card	A JSON business card each agent publishes at `/.well-known/agent-card.json`: its name, URL and skills. Other agents read it to discover what this agent can do
Task	One piece of work you send to an agent. It moves through states such as submitted, working, input-required and completed
Message	A turn of conversation inside a task (user or agent)
Artifact	The result the agent hands back, for example a table or a markdown report
Transport	JSON-RPC 2.0 over HTTP, with streaming updates over SSE when you want them

A2A is about agents talking to agents. **Tools** are how each agent talks to its own data. You need both.

What you must hand in

1. **Three agents**, each a separate A2A server on its own port: two specialists and one orchestrator (the client agent that talks to the user).

2. **Agent Cards** for the specialists, with at least two skills each. The orchestrator must find them by fetching the cards, not by hard-coding what they can do.
3. **Tools on real data**: at least two tools per specialist, working on the CSV files loaded into SQLite or pandas. Use read-only, parameterised queries.
4. **Run the six questions** for your set end to end and save each final answer as a markdown artifact.
5. **Architecture diagram** showing the agents, their ports, their tools and the direction of every call.
6. **Monitoring and observability**: see the last section of this document.
7. **README plus a 3 to 5 minute screen recording.**

Ground rules

- Agents may only share information by sending A2A messages. No importing each other's code and no shared global variables.
- Numbers in the final answer must come from tools, not from the LLM's memory.
- If a specialist is down, the orchestrator must say so clearly and not make up an answer.
- Keep API keys in a .env file.

Tools you can use

What you need	Easy choice	Good to know
A2A	Official Python SDK: `pip install "a2a-sdk[http-server]" uvicorn httpx`	Follow the Python quickstart on a2a-protocol.org for the version you install, because the SDK has changed between versions. Many old samples serve the card at `/.well-known/agent.json`, but current versions use `/.well-known/agent-card.json`
LLM, free to start	NVIDIA Nemotron on build.nvidia.com: `nvidia/nemotron-3-nano-30b-a3b` (fast) or `nvidia/nemotron-3-super-120b-a12b` (stronger tool calling)	OpenAI-compatible API at `https://integrate.api.nvidia.com/v1`
Embeddings (optional)	NVIDIA embedding model `nvidia/llama-3.2-nv-embedqa-1b-v2`	Handy if an agent should look up skills or notes by meaning

What you need	Easy choice	Good to know
Azure	Azure OpenAI or AI Foundry for the model, Azure Container Apps to host the three servers, Application Insights for logs	Optional, local ports are enough to pass
OpenRouter	`https://openrouter.ai/api/v1`	Backup model route
Framework	LangChain for tools, LangGraph for the orchestrator's flow	Optional, plain Python also works
Data	SQLite (built into Python) or pandas	Load the CSVs once at start-up

The easy path in seven steps

1. Load the CSV files into SQLite and write the tool functions. Test them with no agents and no LLM.
2. Follow the SDK quickstart and get a hello-world agent running on one port. Check that `curl http://localhost:9001/.well-known/agent-card.json` returns the card.
3. Turn that hello-world into your first specialist: Agent Card with skills, plus an executor that calls the LLM with your tools.
4. Copy it for the second specialist on another port.
5. Build the orchestrator: fetch both cards, let the LLM decide which agent gets which sub-task, send the tasks, collect the artifacts, write the final answer.
6. Run the six questions and fix what breaks.
7. Add tracing across all three agents and build the dashboard.

Set A: RetailPulse sales analytics team

The story. RetailPulse is a chain of eight stores across India. The head of retail wants to ask plain-English questions about a year of sales and get answers backed by numbers, plus a warning when something looks odd.

Your job. Build three A2A agents that answer those questions together.

Agent	Port	Job	Tools it must have
Data Agent	9001	Owns the sales database and answers data questions	`describe_schema()`, `run_sql(query)` (read-only, SELECT only), `top_n(metric, dimension, n)`
Insight Agent	9002	Finds patterns in a series of numbers it is given	`detect_anomalies(series)` using a z-score, `trend_slope(series)`, `forecast_next(series)` using a straight line
Orchestrator	9000	Talks to the user, discovers both agents from their cards, splits the question, merges the answers	Calls the other two through A2A

Data you get

`stores.csv` has 8 rows and `sales.csv` has 480 rows: 8 stores, 12 months, 5 categories.

store_id	city	format
S01	Bengaluru	Mall
S02	Chennai	Highstreet
S03	Hyderabad	Mall

row_id	store_id	category	month_no	units_sold	revenue_inr
O00001	S01	Electronics	1	75	164526
O00002	S01	Fashion	1	94	189328
O00003	S01	Grocery	1	80	180335

The six questions your team must answer

1. Which store earned the highest total revenue for the year, and how much?
2. For each store, which category earns the most?

3. Is there any store, category and month where revenue looks unusually high or low? Explain why you think so.
4. Is any store losing revenue steadily month after month?
5. If the trend continues, what is the chain's expected total revenue next month?
6. Write a five-line executive summary for the head of retail.

How the agents should cooperate

For question 3, the Orchestrator asks the Data Agent for the monthly revenue series of each store and category, then sends those series to the Insight Agent, which returns the suspicious points. The Orchestrator explains the result in plain language. Question 4 works the same way using the trend tool.

Hints

- Build the monthly series with a SQL `GROUP BY store_id, category, month_no`. Then the z-score is computed inside each series (12 points), not across the whole table.
- A z-score above 3 is a reasonable first threshold. Try 2.5 and see what changes.
- Make the Insight Agent's skills generic (it takes any list of numbers). That is what makes it reusable and is the whole point of A2A.
- Block anything that is not a SELECT in `run_sql`.
- The trainer has an answer key. If your answers to questions 1, 3 and 4 differ from what the data really says, debug the tools, not the prompt.

Set B: StockSense supply chain team

The story. A manufacturer keeps 30 spare parts in stock. Some will run out before the supplier can deliver a new batch. The planner wants a team of agents that spots those parts early and says exactly how many to order from whom.

Your job. Build three A2A agents. The trick is that the stock data and the supplier data are owned by **different agents**, so they have to cooperate to answer anything useful.

Agent	Port	Owns	Tools it must have
Inventory Agent	9001	`inventory.csv`	`get_stock(sku_id)`, `days_of_cover(sku_id)`, `list_skus_by_supplier(supplier_id)`

Agent	Port	Owns	Tools it must have
Supplier Agent	9002	`suppliers.csv`	`get_supplier(supplier_id)`, `lead_time(supplier_id)`, `min_order_qty(supplier_id)`
Planner (orchestrator)	9000	Nothing, it only coordinates	Discovers both cards, combines their answers, writes the plan

Data you get

`inventory.csv` has 30 rows and `suppliers.csv` has 5 rows.

sku_id	item_name	stock_on_hand	avg_daily_demand	supplier_id	open_po_qty
SKU101	Part-01	400	35	SUP5	200
SKU102	Part-02	20	8	SUP4	200
SKU103	Part-03	400	8	SUP4	200

supplier_id	supplier_name	reliability_score	lead_time_days	min_order_qty
SUP1	Apex Components	0.95	5	100
SUP2	Zenith Parts	0.88	9	200
SUP3	Omni Traders	0.80	14	50

`open_po_qty` is stock that has already been ordered and is on its way.

Business rules

```
\text{days of cover} = \frac{\text{stock on hand} + \text{open PO qty}}
{\text{average daily demand}}
```

- A SKU is **AT_RISK** when its days of cover is **less than** its supplier's lead time in days. Otherwise it is OK.
- For an AT_RISK SKU, the shortfall is demand × (lead time + 14) − stock on hand − open PO qty. This covers the lead time plus a 14-day buffer.
- The suggested order is the shortfall **rounded up to a multiple of the supplier's minimum order quantity**. OK SKUs get 0.

The six questions your team must answer

1. Which SKUs are AT_RISK, and how many days of cover does each have?
2. For each AT_RISK SKU, what quantity should be ordered?
3. What is the total quantity to order from each supplier?
4. Which supplier has the most AT_RISK SKUs, and what is its reliability score?
5. Which AT_RISK SKUs depend on a supplier with reliability below 0.85, and what would you recommend?
6. Write a five-line summary for the planning manager.

Hints

- The Inventory Agent cannot decide AT_RISK alone because lead time lives with the Supplier Agent. Let the Planner ask the Inventory Agent for days of cover, then ask the Supplier Agent for lead time, then compare.
- Do the maths in tool code with `math.ceil`, not in the prompt. LLMs are unreliable at rounding.
- Make unknown SKUs or suppliers return a clear error message, and let the Planner handle it politely. If you want to try it, the A2A `input-required` task state is the right way to ask for a corrected SKU.
- The trainer has an answer key. Check question 2 against it before you write the summary.

Set C: AuditAssist expense audit team

The story. A company's finance team checks employee expense claims by hand and always finds problems too late. Auditors want three cooperating agents that go through every claim, flag the ones that break policy, and explain the damage in plain words.

Your job. Build three A2A agents. The policy checker must never read raw expenses directly. It only judges what the orchestrator sends it.

Agent	Port	Job	Tools it must have
Expense Data Agent	9001	Owns `expenses.csv` and `employees.csv`	`get_expenses(filters)`, `get_employee(emp_id)`, `find_duplicate_candidates()`

Agent	Port	Job	Tools it must have
Policy Agent	9002	Owns `policy_limits.csv` and the rules	`get_limit(grade, category)`, `check_expense(expense, employee)`
Audit Orchestrator	9000	Runs the audit, builds the output, writes the summary	Discovers both cards and coordinates them

Data you get

`employees.csv` has 20 rows, `expenses.csv` has 155 rows and `policy_limits.csv` has 15 rows.

expense_id	emp_id	category	vendor	amount_inr	receipt_attached
EXP0001	E004	Lodging	Amazon Business	2312	Y
EXP0002	E001	Client Entertainment	Taj Hotels	12097	Y
EXP0003	E001	Travel	Zomato	20666	Y

emp_id	name	department	grade
E001	Diya Iyer	Engineering	G1
E002	Manoj Sharma	HR	G3
E003	Karthik Kumar	Engineering	G2

Policy limits per claim, in rupees (the full table is in `policy_limits.csv`):

Grade	Travel	Meals	Lodging	Software	Client Entertainment
G1	15000	1500	6000	8000	10000
G2	22500	2250	9000	12000	15000
G3	30000	3000	12000	16000	20000

Business rules

An expense can carry more than one flag.

- **OVER_LIMIT**: the amount is above the limit for the employee's grade and the expense category.
- **MISSING_RECEIPT_OVER_1000**: `receipt_attached` is N and the amount is above 1,000.
- **DUPLICATE**: the same `emp_id`, `vendor`, `category` and `amount_inr` already appeared in an earlier expense. Flag only the later one and name the original.

The six questions your team must answer

1. How many expenses are flagged, split by flag type?
2. Which five employees have the highest total flagged amount?
3. Which department has the highest flagged amount?
4. List every duplicate with its original expense ID.
5. What is the total amount claimed without a receipt above 1,000?
6. Write a short summary for the finance head with recommended actions.

Also save every flagged expense, with its flags, into `flagged_expenses.csv`.

Hints

- The Orchestrator fetches an expense and its employee from the Data Agent, then sends both to the Policy Agent. That keeps each agent's data private to itself.
- Compare amounts as numbers, not text, and remember the limit depends on **both** grade and category.
- Duplicate detection is a good job for the Data Agent's tool: group by the four columns and keep the first as the original.
- Your six answers must reconcile. For example, question 1's count should match the number of rows in `flagged_expenses.csv`.

Set D: EduTrack student risk team

The story. A school with 40 students in one batch wants to spot early who needs help, before the second-term results turn into a crisis. A counsellor cannot read 200 mark entries by hand. Three cooperating agents can.

Your job. Build three A2A agents that find at-risk students and prepare a kind, useful note for the counsellor.

Agent	Port	Job	Tools it must have
Student Data Agent	9001	Owns `students.csv`, `scores.csv` and `attendance.csv`	`get_student_profile(student_id)`, `term_average(student_id, term)`, `subject_drops(student_id)`
Risk Agent	9002	Owns the risk rules	`evaluate_risk(profile)`, `summarise_class(list_of_results)`
Advisor (orchestrator)	9000	Runs the whole check, asks the LLM for counsellor notes, writes the report	Discovers both cards and coordinates them

Data you get

`students.csv` has 40 rows, `scores.csv` has 200 rows (each student has five subjects) and `attendance.csv` has 40 rows.

student_id	name	section
ST001	Rohan Iyer	B
ST002	Karthik Menon	A
ST003	Ananya Reddy	A

student_id	subject	term1_marks	term2_marks
ST001	Maths	44	43
ST001	Science	66	40
ST001	English	45	51

student_id	attendance_percent
ST001	64
ST002	93

student_id	attendance_percent
ST003	89

Business rules

- **LOW_AVG**: the average of the five term 2 marks is below 50.
- **LOW_ATTENDANCE**: attendance is below 75 percent.
- **SHARP_DROP**: at least one subject where term 2 is 15 or more marks lower than term 1.
- Risk level: two or more flags is **HIGH_RISK**, one flag is **WATCH**, no flags is **ON_TRACK**.

The six questions your team must answer

1. How many students are HIGH_RISK, WATCH and ON_TRACK?
2. List every HIGH_RISK student with the flags that triggered.
3. Which section (A, B or C) has the most students who are WATCH or HIGH_RISK?
4. Which subject has the most SHARP_DROP cases?
5. For each HIGH_RISK student, write a counsellor note of at most 60 words.
6. Write a short summary for the principal.

Save one row per student, with the flags and level, in `risk_report.csv`.

Hints

- Compute averages and drops in tool code. Send the LLM only the final flags and a few facts, so it cannot miscalculate.
- These are school students, so **send only student IDs and the needed numbers to the LLM, never names**. Put names back in code when you write the report.
- Notes should be supportive and specific (for example, which subject dropped), and should never label a child as weak or lazy.
- Check that your question 1 counts add up to 40.

Monitoring, observability and marking

With three agents, the hard question is not 'did it work' but 'which agent did what, and where did it go wrong'. Add observability across all three agents and show it working.

What to record

Signal	Why it matters
One **trace id** that follows a question from the Orchestrator through every agent and tool	Lets you see the whole journey of one question in one place
A2A task id and state changes (submitted, working, completed, failed) per agent	Shows where a task stalled or failed
Tool calls with arguments, result size and latency	Finds slow or failing tools
Tokens in and out and estimated cost per agent	Shows which agent is expensive
Errors, timeouts and retries between agents	The early warning that one agent is down

For the trace id, pass the W3C `traceparent` header on every call from one agent to the next and read it on the receiving side. The Python A2A SDK also has an optional OpenTelemetry extra you can try.

How to do it, the easy way

- **Simplest:** each agent writes one JSON line per event to its own log file, always including the trace id, the agent name and the task id. A small script joins the three files into one timeline.
- **OpenTelemetry with Jaeger:** run the Jaeger all-in-one container, send traces to it, and you get a waterfall picture of Orchestrator, Data Agent and the second specialist for each question.
- **Langfuse or LangSmith:** trace the LLM and tool steps inside each agent.
- **Azure:** send OpenTelemetry data to Application Insights if you host the agents on Azure.

The dashboard (one page)

Show tasks per agent, success rate, p95 latency per agent, tool failure rate, tokens and cost per agent, and the trace waterfall for one question, with the three agents clearly visible.

Alerts (at least two)

For example: an agent's health check fails, p95 latency of any agent above a limit you choose, tool failure rate above 5 percent, or an Orchestrator answer produced without a successful call to a specialist. Show how the alert fires.

Prove it works: the failure drill

Stop one specialist while a question is running. Show that the Orchestrator reports the problem honestly, that the trace shows where it failed, and that your alert fires. Then restart the specialist and show it recovering.

How you will be marked

Area	Marks
Agent Cards and real A2A communication (discovery, tasks, artifacts)	20
Tools on the data, correct and safe (read-only, parameterised)	20
Correct answers to the six questions	20
Architecture diagram and agent design	10
Monitoring and observability (cross-agent trace, dashboard, alerts, failure drill)	20
Code quality, README and demo	10
Total	**100**